

TD2 : Design Patterns

Exercice n°1:

Dans ce projet, on trouve deux classes :

- une classe Program1 qui se contente, lorsqu'on appelle go, d'afficher un message (dans la réalité, un traitement particulier aurait lieu), et
- une classe Client qui appelle Program1.

```
public class Client
{
    public static void main1()
    {
        Program1 p = new Program1();
        System.out.println("Je suis le main1");
        p.go();
    }

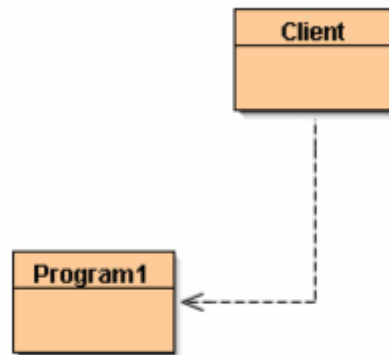
    public static void main2()
    {
        Program1 p = new Program1();
        System.out.println("Je suis le main2");
        p.go();
    }

    public static void main3()
    {
        Program1 p = new Program1();
        System.out.println("Je suis le main3");
        p.go();
    }
}
```

```
public class Program1
{
    public Program1()
    {
        // Le constructeur ne fait rien
    }

    public void go()
    {
        System.out.println("Je suis le traitement 1");
    }
}
```

le diagramme de classe ressemble à ceci

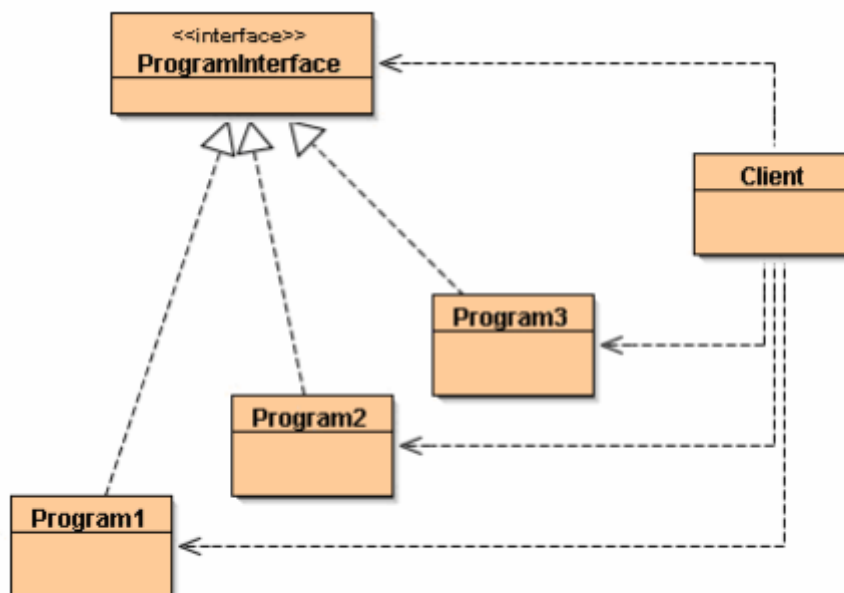


- 1) On souhaite rajouter deux classes Program2 et Program3 à notre projet. Ces classes afficheront à l'écran le même message que Program1, excepté qu'elles y feront apparaître leur numéro de programme (2 ou 3). Ajoutez ces classes (en dupliquant et modifiant le code de Program1).

Dans le client, on souhaite modifier le code des fonctions main pour que, selon le paramètre entier qui leur est passé en argument (1, 2, ou 3), ces dernières lancent le traitement du Program1, Program2 ou Program3.

Pouvez-vous réaliser cette fonctionnalité ?

en codant de la bonne manière, le diagramme de classe que vous devriez obtenir:



Qu'est que vous remarquez?(quels sont les problèmes posés par une telle solution)

2) **Appliquer le pattern Factory**

Déléguer la création des objets de type Program à une classe dont le nom sera ProgramFactory

- ✓ Elaborer le diagramme de classes répondant à ce besoin
- ✓ Donner le code des différentes entités participantes à la réalisation du programme(classes, interfaces)
- ✓ Pouvez-vous ajouter un Program4 ? Cela a-t-il été compliqué à mettre en place ? Avez-vous dû modifier le code du Client ?

Exercice n°2

Le système de vente de véhicules gère des véhicules fonctionnant à l'essence et des véhicules fonctionnant à l'électricité.

Cette gestion est confiée à l'objet **Catalogue** qui crée de tels objets. Pour chaque produit, nous disposons d'une classe abstraite, d'une sous-classe concrète décrivant la version du produit fonctionnant à l'essence et d'une sous-classe décrivant la version du produit fonctionnant à l'électricité.

Questions

1) proposer une conception à ce problème en adoptant le pattern Factory

- ✓ donner le diagramme de classes tout en expliquant le rôle de chacune des composantes
- ✓ implémenter la solution

2) On envisage d'ajouter de nouvelles familles de produits en l'occurrence des Véhicules ainsi que des scooter Diesel

- ✓ Apporter les modifications nécessaires à la solution précédente afin de répondre à ce besoin. Que remarquez-vous?
- ✓ Proposer une solution en utilisant le pattern Abstract-Factory.

Exercice n°3

Vous êtes amateur de pêche en mer et vous ne pouvez pratiquer votre "hobby" qu'avec une bonne météo. Une bonne météo est définie par un faible vent (8km/H et moins) et des vagues de 0,3m et moins.

D'habitude, vous consultez l'application windy chaque jour, voir chaque heure mais vous en avez assez car ceci vous consomme beaucoup de temps et vous avez des tâches à réaliser.

Vous avez décidé de mettre ne place votre propre système sachant que vous disposez des classes suivantes :

Meteo (date, vitesseVent, hauteurVague)

User (id, name, eMail, telMobile)

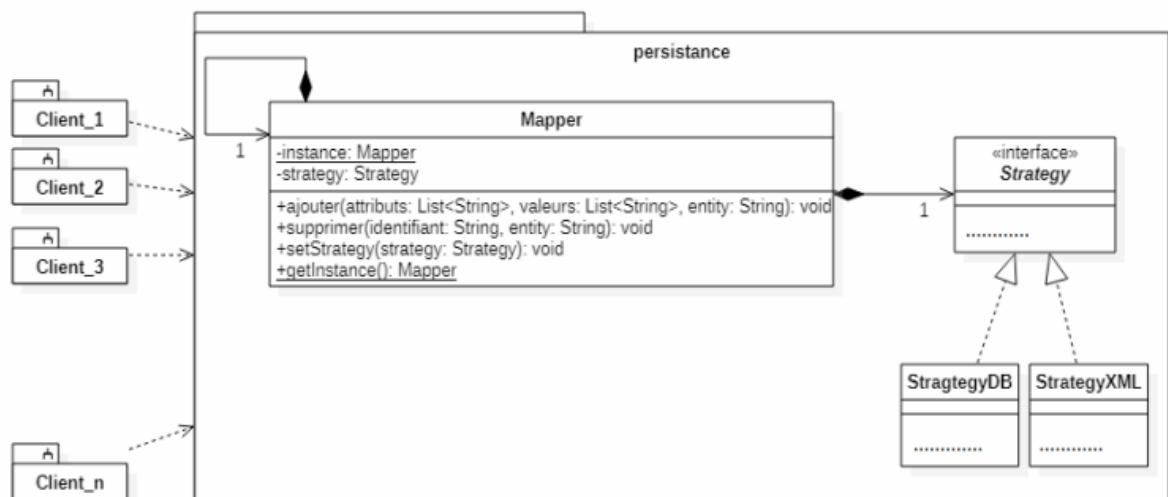
Questions :

1/ Proposer une solution (Classe et classe de Test) tel que le système vous enverra ainsi qu'à vos amis Hichem et Riadh un eMail quand les prévisions seront favorables.

2/ Améliorer la solution si nous voulons que le système enverra un SMS ou un eMail selon les choix de l'utilisateur. Ce choix sera configuré quelque part dans la classe utilisateur ou dans une autre class de configuration.

Exercice 4 :

Soit le diagramme de classe d'un module de persistance de données dans une application de gestion d'un entrepôt de logistique, le module permettra d'enregistrer les colis et/ou les clients dans une base de données relationnelle ou un fichier xml. Le Mapper est l'objet responsable des actions de persistance (on ne garde que l'ajout et la suppression dans notre cas), c'est un objet de type singleton qui doit proposer deux stratégies de persistance.



Questions :

1. Rappelez l'utilité des patterns Singleton et Stratégie. Donnez pour chaque pattern : l'objectif (quand l'utiliser) et une description.

2. Donnez les squelettes de l'interface Strategy et des deux implémentations StrategyDB et StrategyXML.

3. Implémentez la Classe Mapper (donnez le code complet) sachant que l'utilisation de la BD est la stratégie par défaut.

4. Donnez un exemple d'utilisation du module (commentez votre code et utilisez les deux modes d'enregistrement).

On suppose que les deux classes Colis et Client sont implémentées :

- Un colis est identifié par un code (String) et il a un poids (double).
- Un Client est identifié par son cin (String), il a un e-mail (String), et un tél (int)